

EXPERIMENT-5: Java standalone CRUD application

Develop a Java standalone application that connects with a database (Oracle/MySQL) and performs CRUD operations on database tables.

Objective : Build a desktop-like Java app with database connectivity and CRUD flow.

Tools : Java, JDBC, MySQL/Oracle, SQL

Outcome : A database-connected Java app capable of creating, reading, updating, and deleting records.

Answer:

Open a JDBC connection, prepare SQL statements for inserts/updates/deletes, and display query results in the UI.

📁 File Structure

week-5/

```
├── StudentCRUDH2.java    (Main program - H2 Database version)
├── StudentCRUD.java      (MySQL version - requires server)
├── RunStudent.bat        (Run H2 version)
├── RunStudentApp.bat     (Choose H2 or MySQL)
├── lib/
|   ├── h2.jar            (H2 Database driver)
|   └── mysql-connector-j.jar (MySQL JDBC driver)
└── studentdb.mv.db       (Database file - auto-created)
```

File Name: StudentCRUDH2.java

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
/**
```

```
 * Student CRUD Application using H2 Embedded Database
```

```
 * No setup required - just run it! Database file is created automatically.
```

```
 */
```

```

public class StudentCRUDH2 {
    // H2 embedded database connection (no server needed!)
    private static final String URL = "jdbc:h2:./studentdb;MODE=MySQL";
    private static final String USER = "sa";
    private static final String PASSWORD = "";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            // Load H2 driver
            Class.forName("org.h2.Driver");

            // Create connection (database file will be created if it doesn't exist)
            Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("✓ Connected to Student Database successfully!");

            // Initialize database tables on first run
            initializeDatabase(connection);

            // Main menu loop
            while (true) {
                System.out.println("\n" + "=".repeat(35));
                System.out.println("    STUDENT MANAGEMENT SYSTEM");
                System.out.println("=".repeat(35));
                System.out.println("1. + Add Student");
                System.out.println("2. 👁 View All Students");
                System.out.println("3. ✎ Update Student");
                System.out.println("4. ✖ Delete Student");
                System.out.println("5. 🏠 Exit");
                System.out.println("=".repeat(35));
                System.out.print("Enter your choice (1-5): ");
            }
        }
    }
}

```

```

int choice = scanner.nextInt();
scanner.nextLine();

switch (choice) {
    case 1:
        addStudent(connection, scanner);
        break;
    case 2:
        viewStudents(connection);
        break;
    case 3:
        updateStudent(connection, scanner);
        break;
    case 4:
        deleteStudent(connection, scanner);
        break;
    case 5:
        connection.close();
        scanner.close();
        System.out.println("\n👋 Thank you for using Student Management System!");
        return;
    default:
        System.out.println("❌ Invalid choice. Please try again.");
}
}
} catch (Exception e) {
    System.err.println("❌ Error: " + e.getMessage());
    e.printStackTrace();
}
}

/**
 * Initialize database tables on first run

```

```

*/
private static void initializeDatabase(Connection connection) throws SQLException {
    String createTableSQL = "CREATE TABLE IF NOT EXISTS students (" +
        "id INT AUTO_INCREMENT PRIMARY KEY, " +
        "name VARCHAR(100) NOT NULL, " +
        "email VARCHAR(100), " +
        "age INT)";

    try (Statement statement = connection.createStatement()) {
        statement.execute(createTableSQL);
        System.out.println("✓ Database tables ready!");
    }
}

```

```

private static void addStudent(Connection connection, Scanner scanner) throws
SQLException {
    System.out.print("\nEnter student name: ");
    String name = scanner.nextLine().trim();

    if (name.isEmpty()) {
        System.out.println("✗ Name cannot be empty!");
        return;
    }

    System.out.print("Enter student email: ");
    String email = scanner.nextLine().trim();

    System.out.print("Enter student age: ");
    int age = 0;
    try {
        age = scanner.nextInt();
        scanner.nextLine();
    } catch (Exception e) {

```

```

scanner.nextLine();
System.out.println(" ❌ Invalid age. Please enter a number.");
return;
}

```

```

String sql = "INSERT INTO students (name, email, age) VALUES (?, ?, ?)";
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
    preparedStatement.setString(1, name);
    preparedStatement.setString(2, email);
    preparedStatement.setInt(3, age);

    int rows = preparedStatement.executeUpdate();
    System.out.println("✓ " + rows + " student added successfully!");
}
}

```

```

private static void viewStudents(Connection connection) throws SQLException {
    String sql = "SELECT * FROM students ORDER BY id";
    try (Statement statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery(sql)) {

        System.out.println("\n" + "-".repeat(70));
        System.out.printf("%-5s %-25s %-30s %-5s\n", "ID", "Name", "Email", "Age");
        System.out.println("-".repeat(70));

        boolean hasRecords = false;
        while (resultSet.next()) {
            hasRecords = true;
            int id = resultSet.getInt("id");
            String name = resultSet.getString("name");
            String email = resultSet.getString("email") != null ? resultSet.getString("email") :
"N/A";
            int age = resultSet.getInt("age");

```

```

        System.out.printf("%-5d %-25s %-30s %-5d%n", id, name, email, age);
    }

    if (!hasRecords) {
        System.out.println("No students found in the database.");
    }
    System.out.println("-".repeat(70));
}
}

```

private static void updateStudent(Connection connection, Scanner scanner) throws
SQLException {

```

    System.out.print("\nEnter student ID to update: ");
    int id = 0;
    try {
        id = scanner.nextInt();
        scanner.nextLine();
    } catch (Exception e) {
        scanner.nextLine();
        System.out.println(" ❌ Invalid ID. Please enter a number.");
        return;
    }

```

// Check if student exists

```

if (!studentExists(connection, id)) {
    System.out.println(" ❌ Student with ID " + id + " not found!");
    return;
}

```

```

System.out.print("Enter new name: ");
String name = scanner.nextLine().trim();

```

```

System.out.print("Enter new email: ");

```

```
String email = scanner.nextLine().trim();
```

```
System.out.print("Enter new age: ");
```

```
int age = 0;
```

```
try {
```

```
    age = scanner.nextInt();
```

```
    scanner.nextLine();
```

```
} catch (Exception e) {
```

```
    scanner.nextLine();
```

```
    System.out.println(" ❌ Invalid age. Please enter a number.");
```

```
    return;
```

```
}
```

```
String sql = "UPDATE students SET name = ?, email = ?, age = ? WHERE id = ?";
```

```
try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
```

```
    preparedStatement.setString(1, name);
```

```
    preparedStatement.setString(2, email);
```

```
    preparedStatement.setInt(3, age);
```

```
    preparedStatement.setInt(4, id);
```

```
    int rows = preparedStatement.executeUpdate();
```

```
    System.out.println("✓ " + rows + " student updated successfully!");
```

```
}
```

```
}
```

```
private static void deleteStudent(Connection connection, Scanner scanner) throws  
SQLException {
```

```
    System.out.print("\nEnter student ID to delete: ");
```

```
    int id = 0;
```

```
    try {
```

```
        id = scanner.nextInt();
```

```
        scanner.nextLine();
```

```
    } catch (Exception e) {
```

```

        scanner.nextLine();
        System.out.println(" ❌ Invalid ID. Please enter a number.");
        return;
    }

    // Check if student exists
    if (!studentExists(connection, id)) {
        System.out.println(" ❌ Student with ID " + id + " not found!");
        return;
    }

    String sql = "DELETE FROM students WHERE id = ?";
    try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
        preparedStatement.setInt(1, id);

        int rows = preparedStatement.executeUpdate();
        System.out.println("✓ " + rows + " student deleted successfully!");
    }
}

/**
 * Helper method to check if a student exists
 */
private static boolean studentExists(Connection connection, int id) throws SQLException {
    String sql = "SELECT COUNT(*) FROM students WHERE id = ?";
    try (PreparedStatement preparedStatement = connection.prepareStatement(sql)) {
        preparedStatement.setInt(1, id);
        try (ResultSet resultSet = preparedStatement.executeQuery()) {
            if (resultSet.next()) {
                return resultSet.getInt(1) > 0;
            }
        }
    }
}

```



```
        return false;
    }
}
```

Folder name: StudentCRUD.java

```
import java.sql.*;
import java.util.Scanner;

public class StudentCRUD {
    private static final String URL = "jdbc:mysql://localhost:3306/studentdb";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Connected to MySQL database successfully.");

            while (true) {
                System.out.println("\n===== Student CRUD Menu =====");
                System.out.println("1. Add Student");
                System.out.println("2. View Students");
                System.out.println("3. Update Student");
                System.out.println("4. Delete Student");
                System.out.println("5. Exit");
                System.out.print("Enter your choice: ");

                int choice = scanner.nextInt();
                scanner.nextLine();
            }
        }
    }
}
```

```

switch (choice) {
    case 1:
        addStudent(connection, scanner);
        break;
    case 2:
        viewStudents(connection);
        break;
    case 3:
        updateStudent(connection, scanner);
        break;
    case 4:
        deleteStudent(connection, scanner);
        break;
    case 5:
        connection.close();
        scanner.close();
        System.out.println("Goodbye!");
        return;
    default:
        System.out.println("Invalid choice. Try again.");
}
}
} catch (Exception e) {
    e.printStackTrace();
}
}

```

```

private static void addStudent(Connection connection, Scanner scanner) throws
SQLException {
    System.out.print("Enter student name: ");
    String name = scanner.nextLine();

    System.out.print("Enter student email: ");

```

```
String email = scanner.nextLine();
```

```
System.out.print("Enter student age: ");
```

```
int age = scanner.nextInt();
```

```
scanner.nextLine();
```

```
String sql = "INSERT INTO students (name, email, age) VALUES (?, ?, ?)";
```

```
PreparedStatement preparedStatement = connection.prepareStatement(sql);
```

```
preparedStatement.setString(1, name);
```

```
preparedStatement.setString(2, email);
```

```
preparedStatement.setInt(3, age);
```

```
int rows = preparedStatement.executeUpdate();
```

```
System.out.println(rows + " student added successfully.");
```

```
}
```

```
private static void viewStudents(Connection connection) throws SQLException {
```

```
String sql = "SELECT * FROM students";
```

```
Statement statement = connection.createStatement();
```

```
ResultSet resultSet = statement.executeQuery(sql);
```

```
System.out.println("\nStudent Records:");
```

```
while (resultSet.next()) {
```

```
    int id = resultSet.getInt("id");
```

```
    String name = resultSet.getString("name");
```

```
    String email = resultSet.getString("email");
```

```
    int age = resultSet.getInt("age");
```

```
    System.out.println("ID: " + id + " | Name: " + name + " | Email: " + email + " | Age: " +  
age);
```

```
}
```

```
}
```

```
private static void updateStudent(Connection connection, Scanner scanner) throws  
SQLException {
```

```
System.out.print("Enter student ID to update: ");  
int id = scanner.nextInt();  
scanner.nextLine();
```

```
System.out.print("Enter new name: ");  
String name = scanner.nextLine();
```

```
System.out.print("Enter new email: ");  
String email = scanner.nextLine();
```

```
System.out.print("Enter new age: ");  
int age = scanner.nextInt();  
scanner.nextLine();
```

```
String sql = "UPDATE students SET name = ?, email = ?, age = ? WHERE id = ?";  
PreparedStatement preparedStatement = connection.prepareStatement(sql);  
preparedStatement.setString(1, name);  
preparedStatement.setString(2, email);  
preparedStatement.setInt(3, age);  
preparedStatement.setInt(4, id);
```

```
int rows = preparedStatement.executeUpdate();  
System.out.println(rows + " student updated successfully.");  
}
```

```
private static void deleteStudent(Connection connection, Scanner scanner) throws  
SQLException {
```

```
    System.out.print("Enter student ID to delete: ");  
    int id = scanner.nextInt();  
    scanner.nextLine();
```

```
    String sql = "DELETE FROM students WHERE id = ?";  
    PreparedStatement preparedStatement = connection.prepareStatement(sql);  
    preparedStatement.setInt(1, id);
```

```
        int rows = preparedStatement.executeUpdate();
        System.out.println(rows + " student deleted successfully.");
    }
}
```

File Name: RunStudent.bat

```
@echo off
REM Student CRUD Application - Just Double-Click to Run!
REM No setup needed - database is automatic!

cd /d "%~dp0"

echo.
echo =====
echo STUDENT MANAGEMENT SYSTEM
echo =====
echo.

java -cp ".;lib\h2.jar" StudentCRUDH2

pause
```

File Name: RunStudentApp.bat

```
@echo off
REM =====
REM STUDENT CRUD - Choose Your Version
REM =====
REM

cd /d "%~dp0"
```

```

echo.
echo =====
echo    STUDENT MANAGEMENT SYSTEM
echo =====
echo.
echo Choose your option:
echo 1. Run with H2 Database (Recommended - No setup needed)
echo 2. Run with MySQL (Requires MySQL Server running)
echo 3. Exit
echo.

```

```

set /p choice="Enter your choice (1-3): "

```

```

if "%choice%"=="1" (
    echo.
    echo Starting with H2 Database...
    echo.
    java -cp ".\lib\h2.jar" StudentCRUDH2
) else if "%choice%"=="2" (
    echo.
    echo Starting with MySQL...
    echo.
    echo Make sure MySQL Server is running on localhost:3306
    echo.
    java -cp ".\lib\mysql-connector-j.jar" StudentCRUD
) else (
    echo Goodbye!
)

```

```

pause

```

File Name : lib/

```

|  └─ h2.ja

```

File name : lib/ mysql-connector-j.jar

➔ This file auto created in you folder studentdb.mv.db (Database file - auto-created)